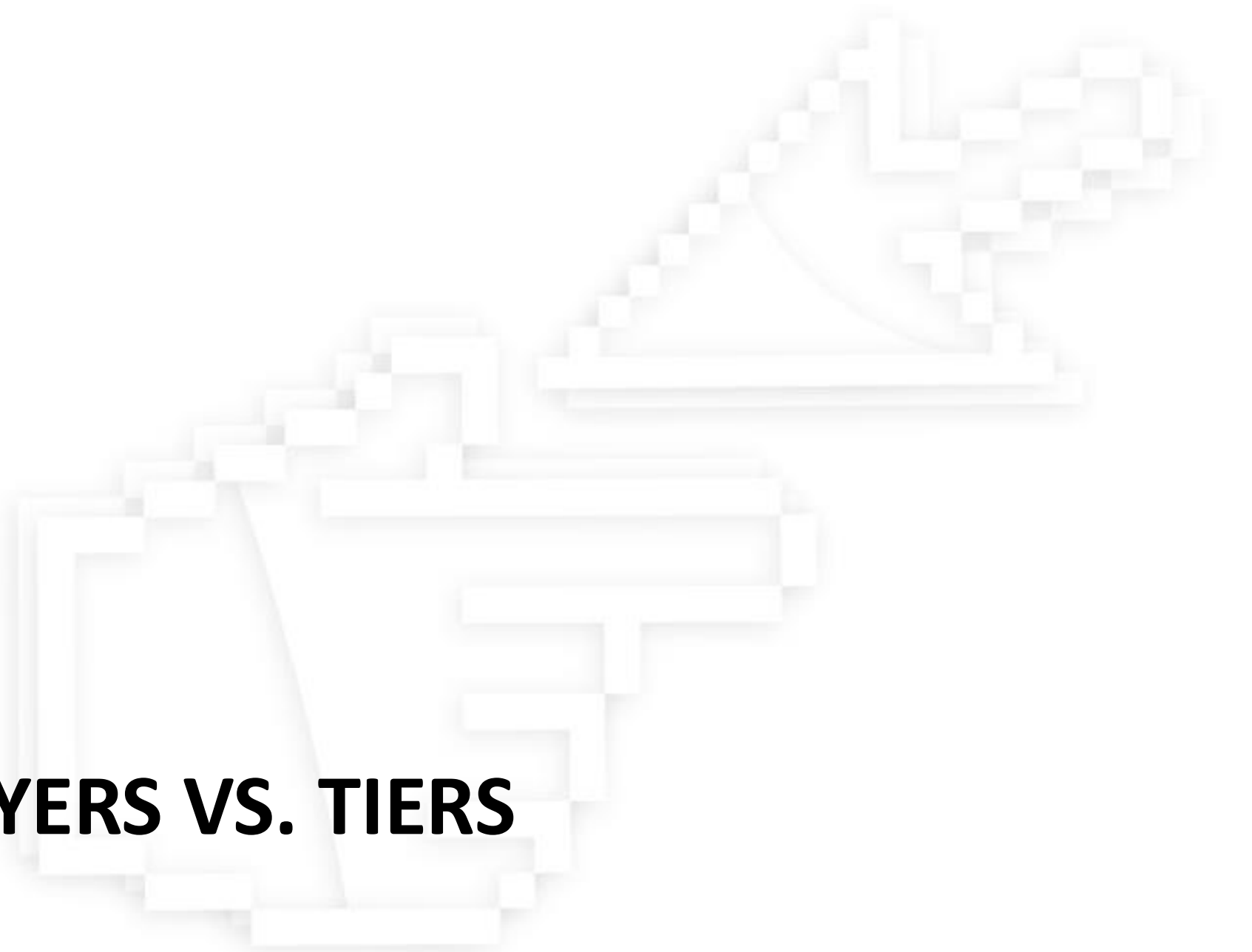




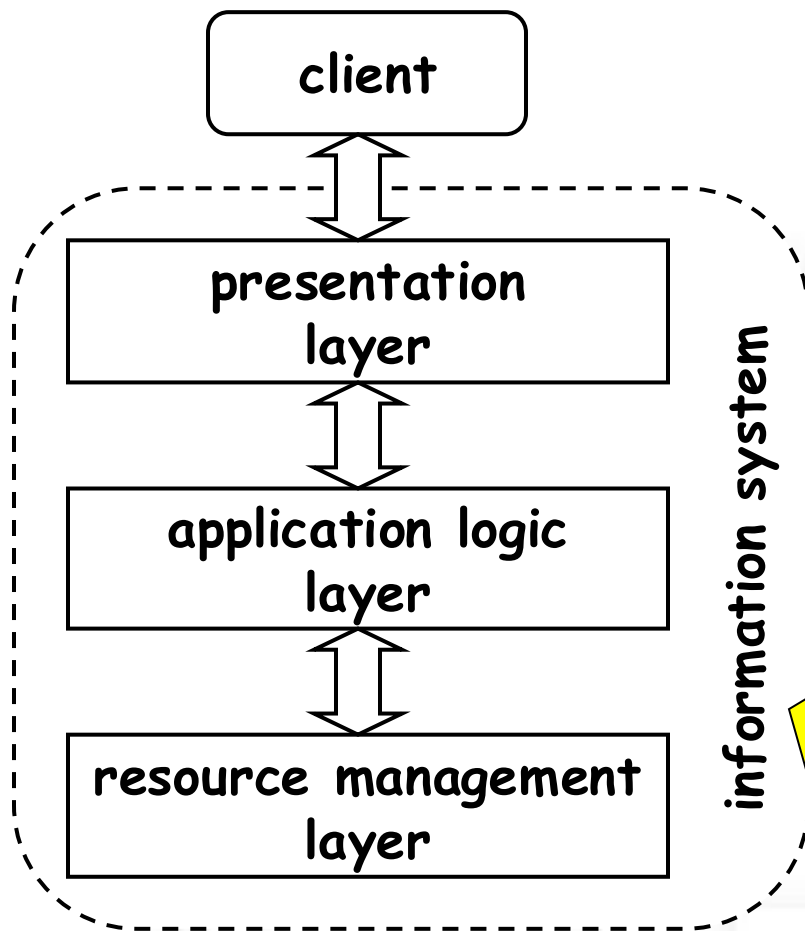
Laboratory of advanced Programming

Distributed programming
Remote Procedure Call (RPC) & Message
Oriented Middleware (MOM)
RPC & MOM in Java: Java RMI & Jakarta
Messaging (formerly JMS)



LAYERS VS. TIERS



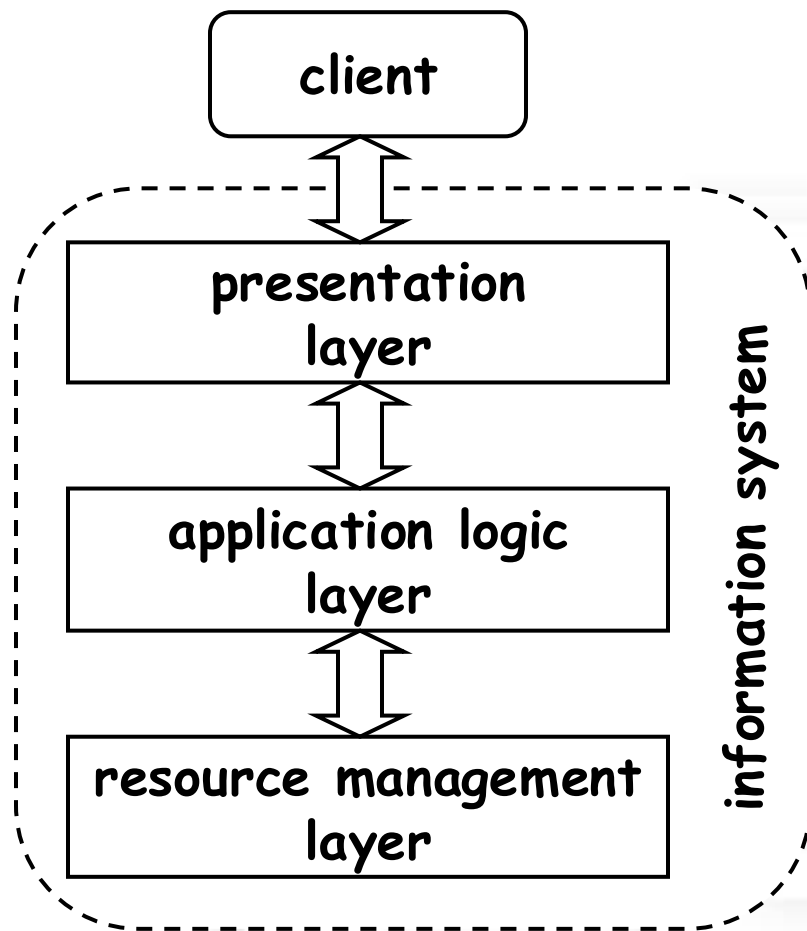


Distributed Information Systems

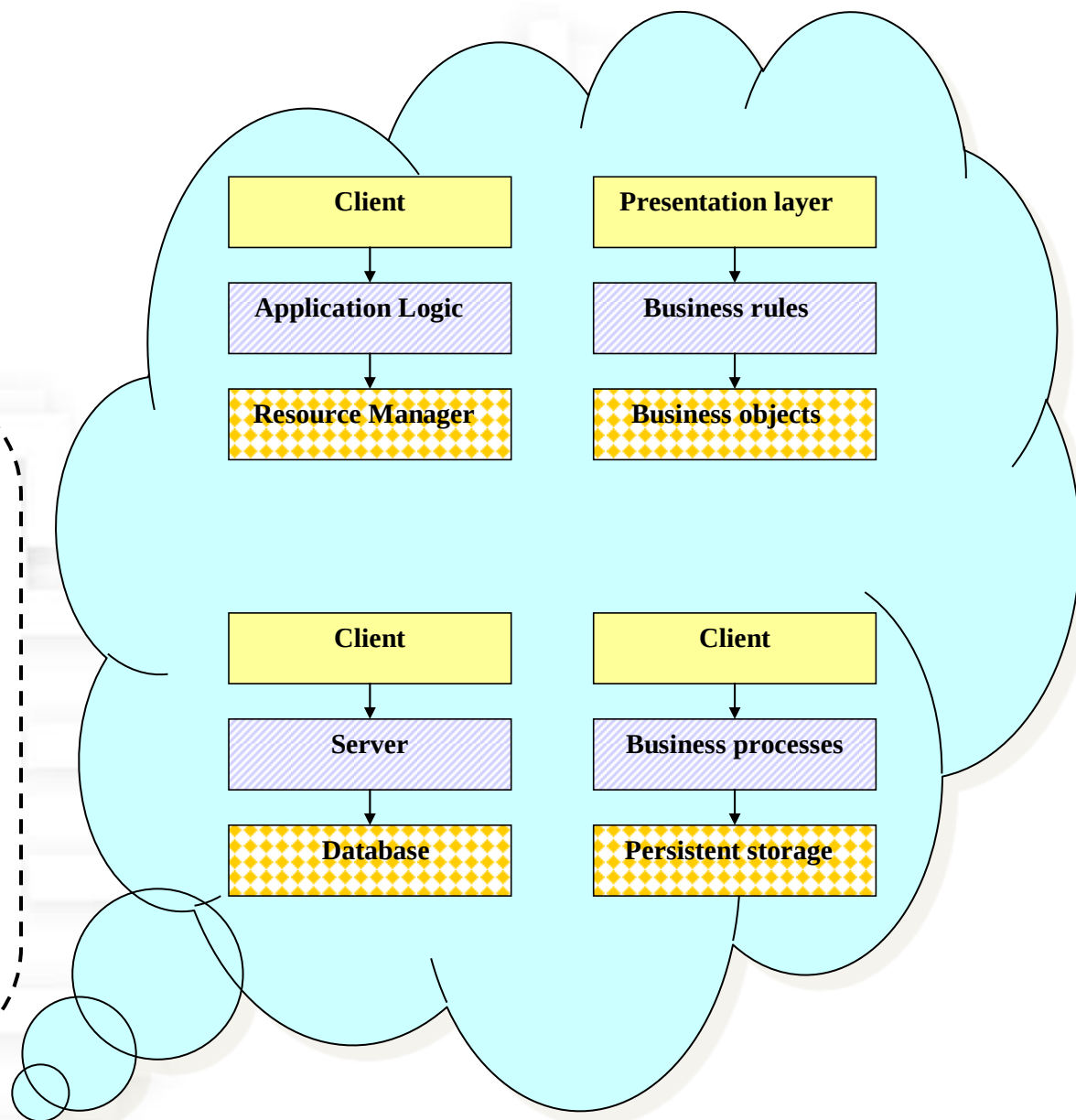
Client is any user or program that wants to perform an operation over the system. Clients interact with the system through a presentation layer

The application logic determines what the system actually does. It takes care of enforcing the business rules and establish the business processes. The application logic can take many forms: programs, constraints, business processes, etc. The resource manager deals with the organization (storage, indexing, and retrieval) of the data necessary to support the application logic. This is typically a database but it can also be a text retrieval system or any other data management system providing querying capabilities and persistence

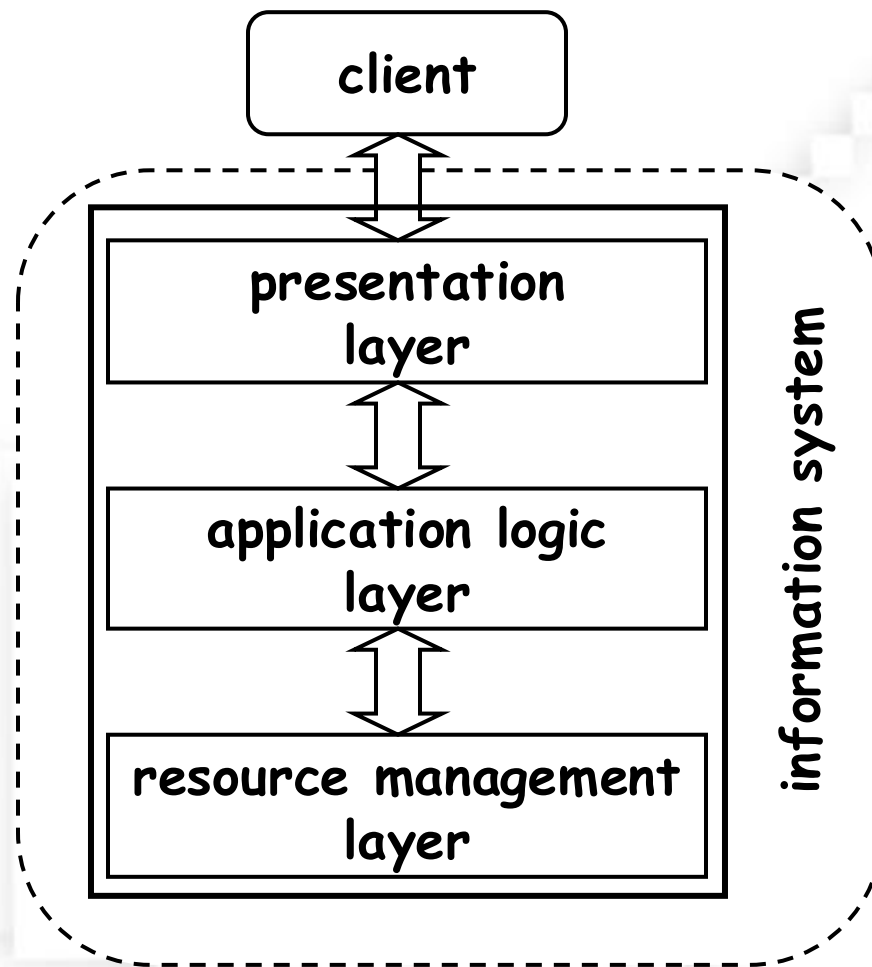




Distributed Information Systems



1-tier Architecture



Distributed Information Systems

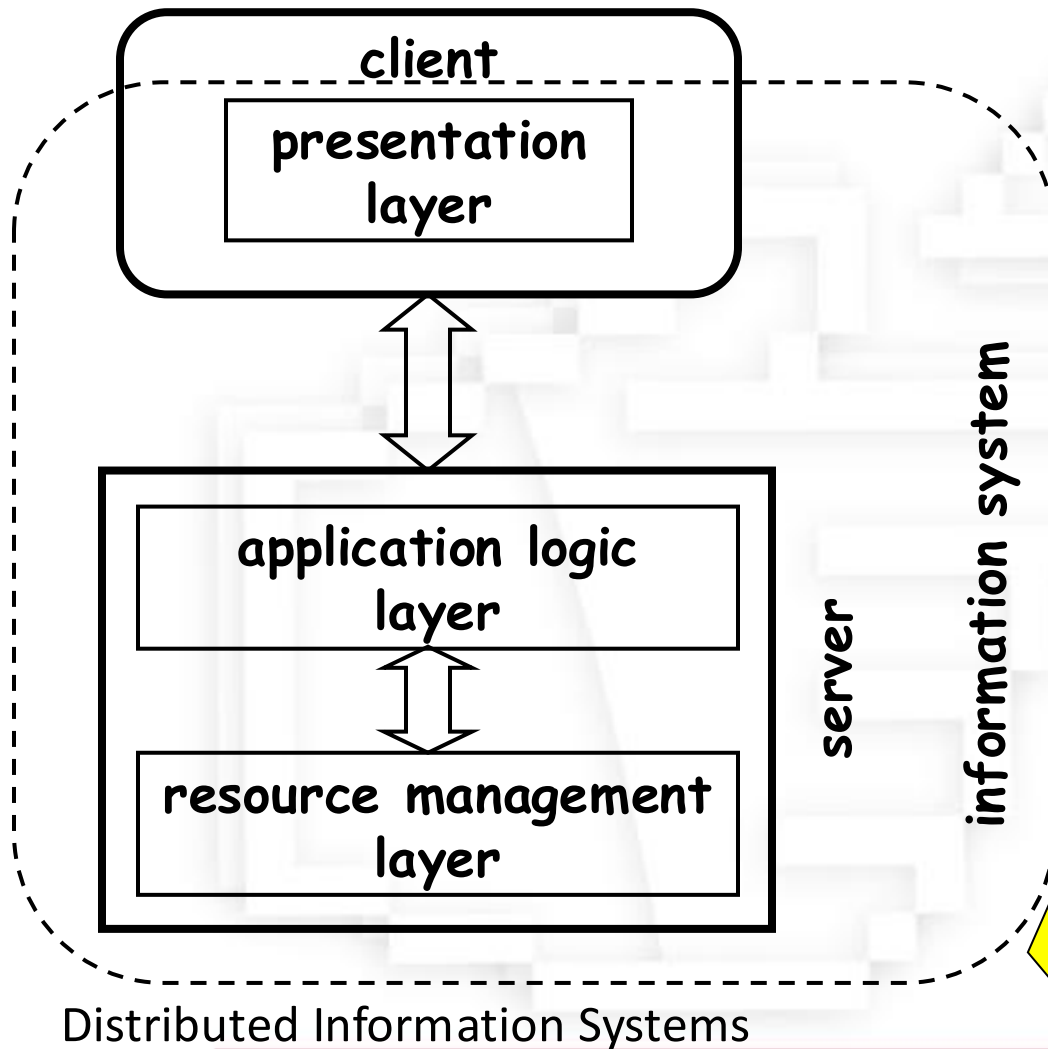
no forced context switches in the control flow (everything happens within the system)

all is centralized, managing and controlling resources is easier

the design can be highly optimized by blurring the separation between layers.



2-tier Architecture



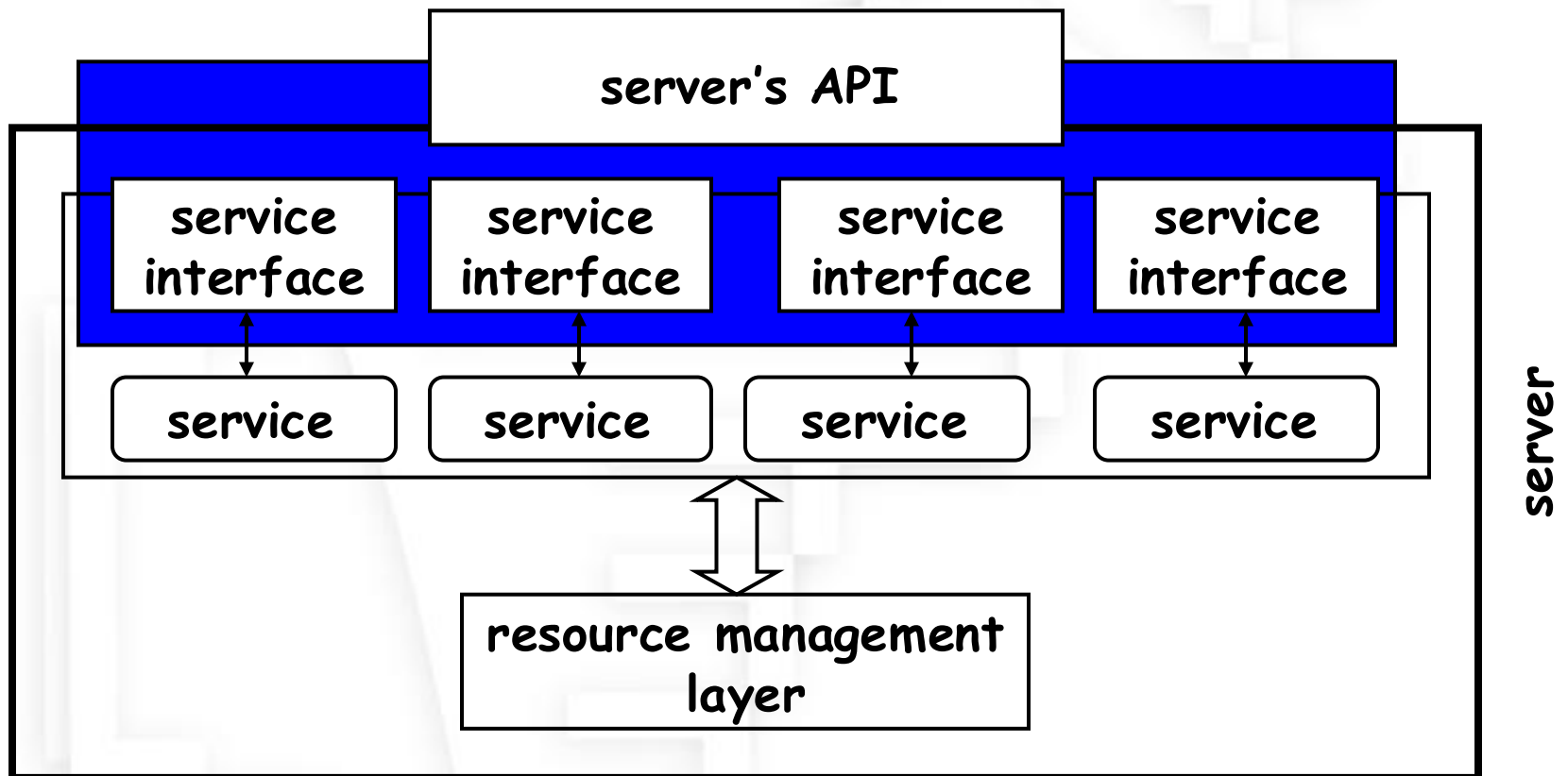
Clients are independent of each other: one could have several presentation layers depending on what each client wants to do. One can take advantage of the computing power at the client machine to have more sophisticated presentation layers. This also saves computer resources at the server machine.

It introduces the concept of API (Application Program Interface). An interface to invoke the system from the outside. It also allows designers to think about federating the systems into a single system.

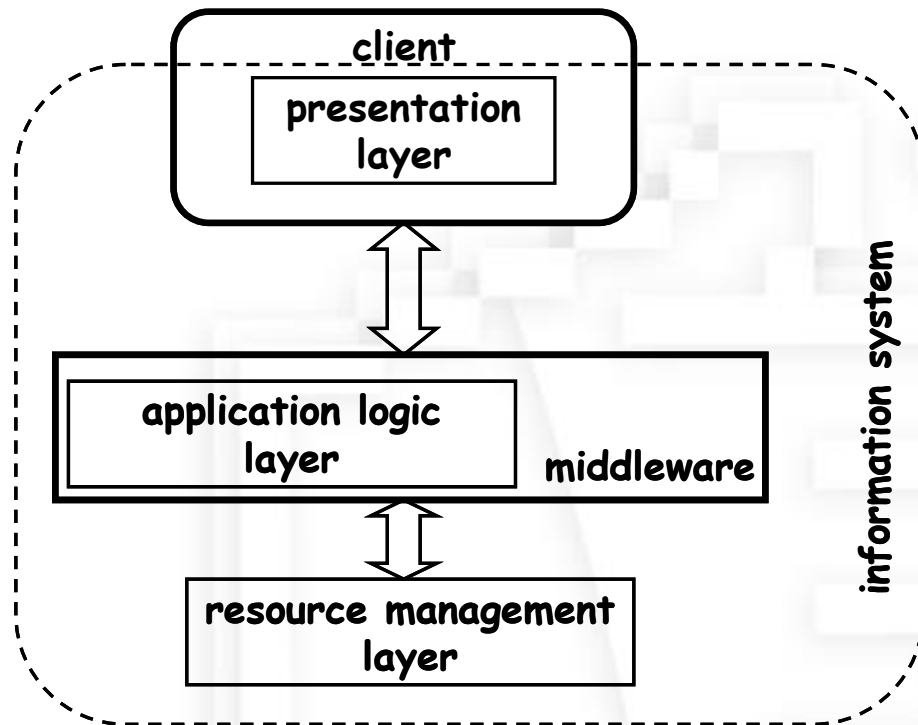
The resource manager only sees one client: the application logic. This greatly helps with performance since there are no client connections/sessions to maintain.



Interfaces and “Services”



3-tier Architecture



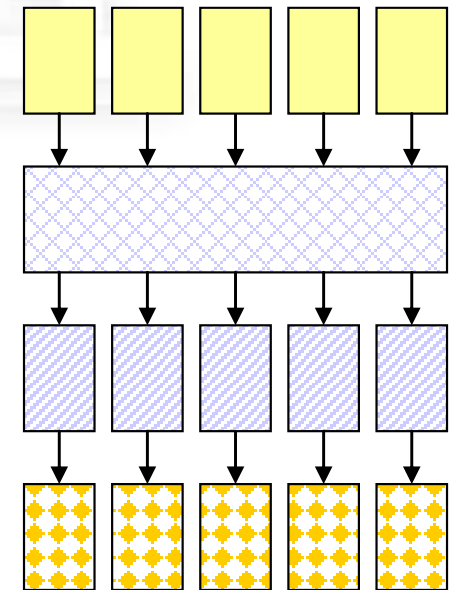
Distributed Information Systems

clients

Middleware or global application logic

Local application logic

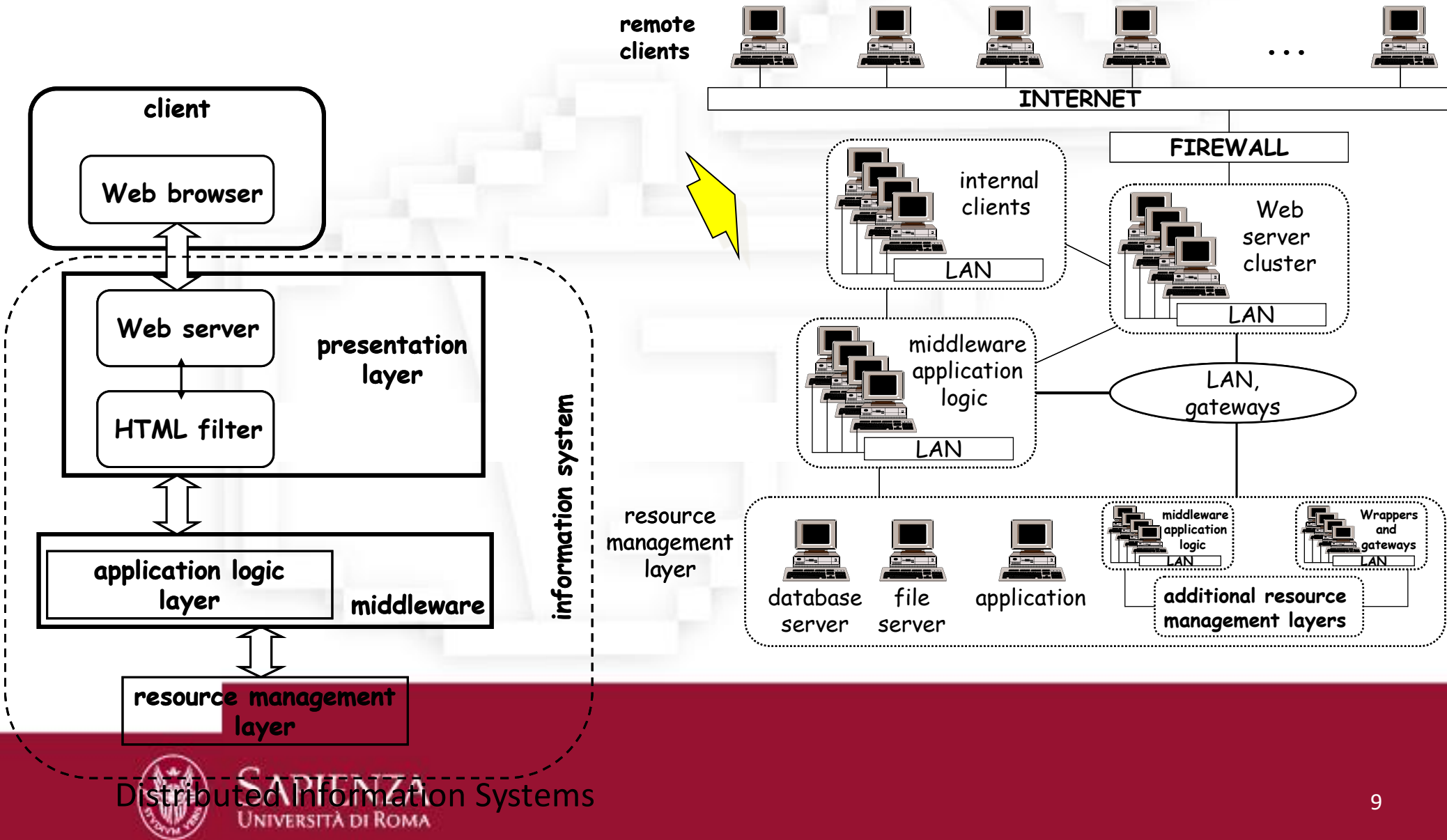
Local resource managers



Middleware is just a level of indirection between clients and other layers of the system. It introduces an additional layer of business logic encompassing all underlying systems.



N-tier Architecture

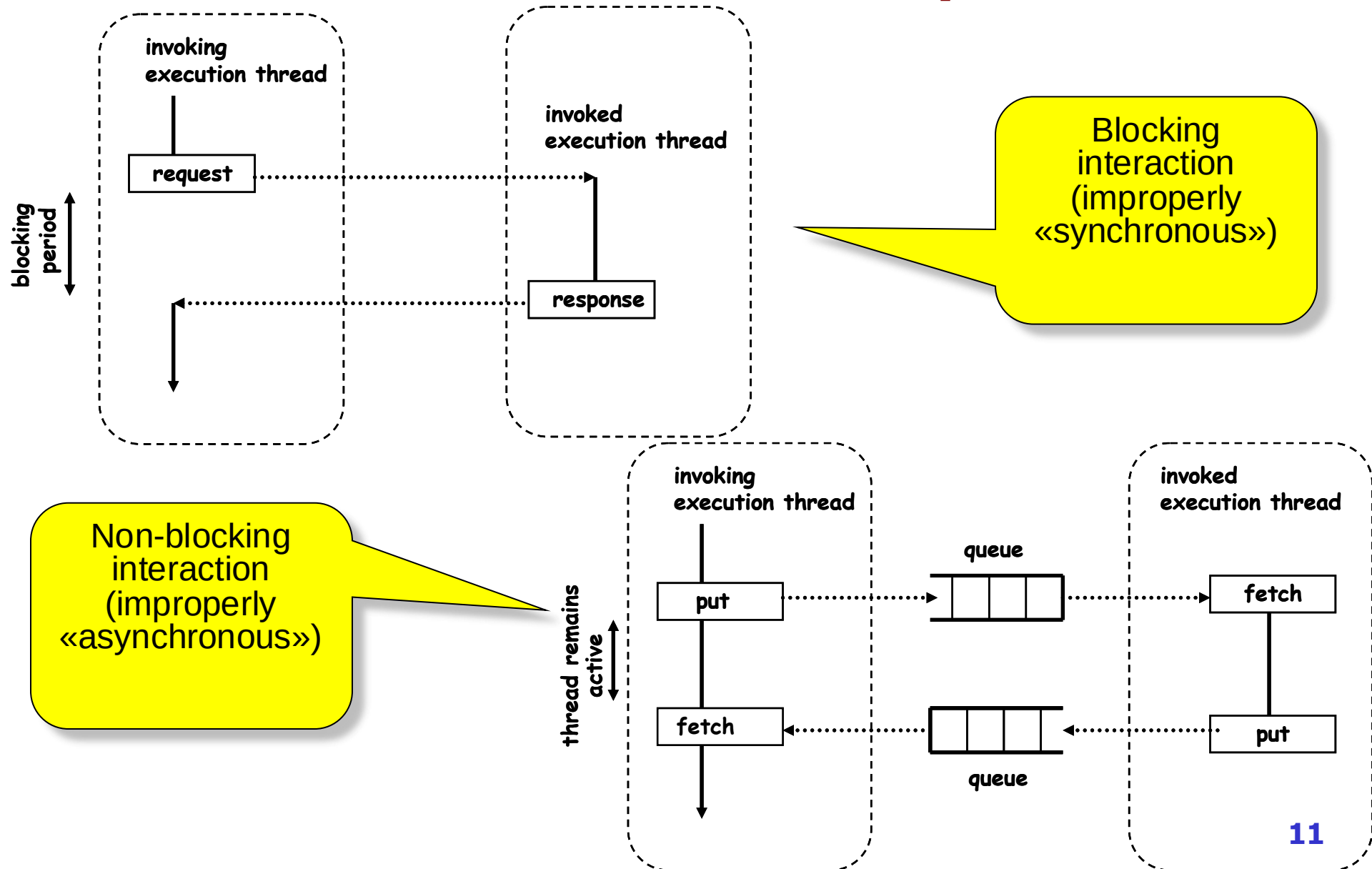




MIDDLEWARE

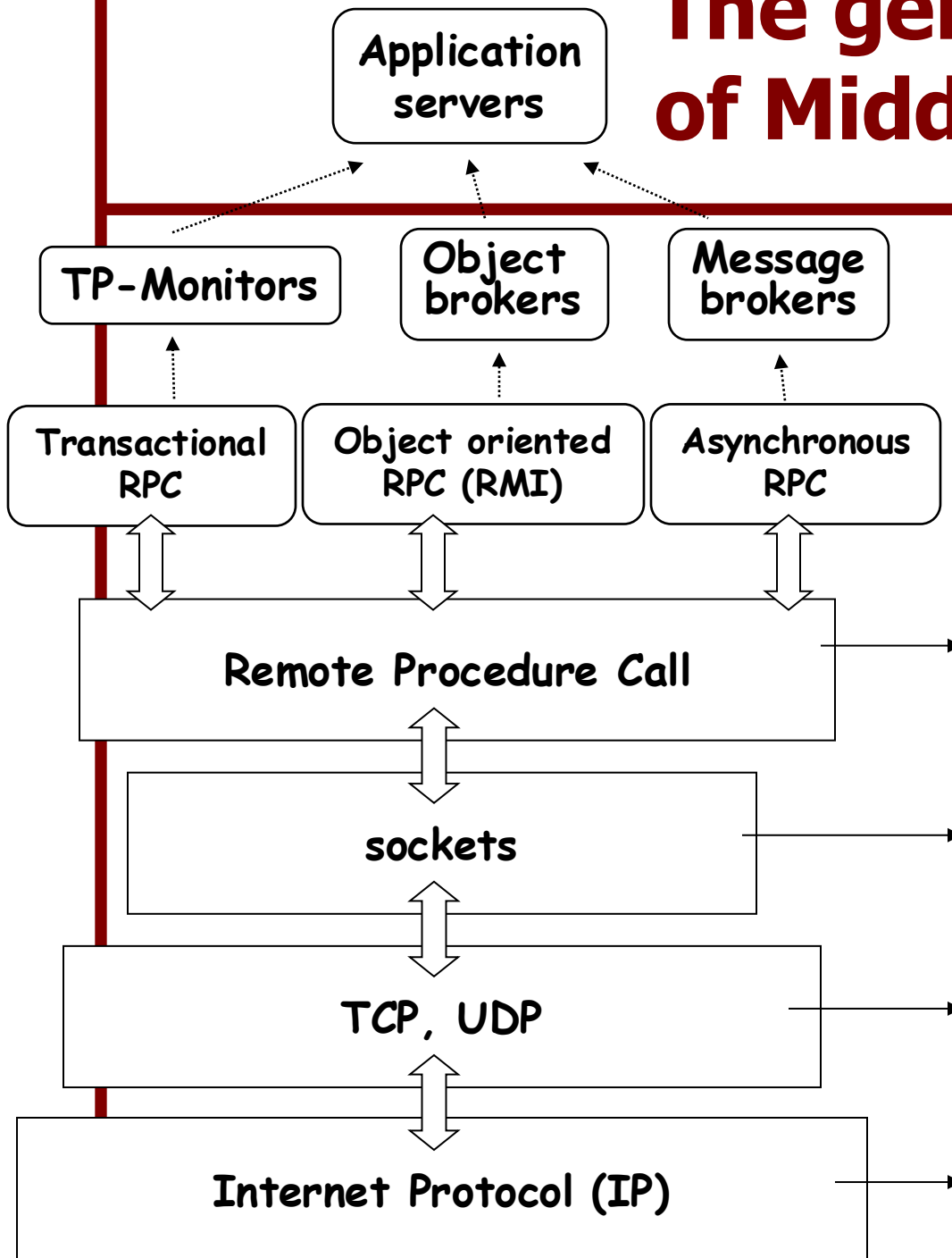


Communication primitives





The genealogy of Middleware



Remote Procedure Call:

hides communication details behind a procedure call and helps bridge heterogeneous platforms

sockets:

operating system level interface to the underlying communication protocols

TCP, UDP:

User Datagram Protocol (UDP) transports data packets without guarantees

Transmission Control Protocol (TCP) verifies correct delivery of data streams

Internet Protocol (IP):

moves a packet of data from one node to another

Understanding Middleware



PROGRAMMING ABSTRACTION

- Intended to hide low level details of hardware, networks, and distribution
- Trend is towards increasingly more powerful primitives that, without changing the basic concept of RPC, have additional properties or allow more flexibility in the use of the concept
- Evolution and appearance to the programmer is dictated by the trends in programming languages (RPC and C, CORBA and C++, RMI and Java, Web services and SOAP-XML)

INFRASTRUCTURE

- Intended to provide a comprehensive platform for developing and running complex distributed systems
- Trend is towards service oriented architectures at a global scale and standardization of interfaces
- Another important trend is towards single vendor software stacks to minimize complexity and streamline interaction
- Evolution is towards integration of platforms and flexibility in the configuration (plus autonomic behavior)



Middleware as a Programming Abstraction

- Programming languages and almost any form of software system evolve always towards higher levels of abstraction
 - hiding hardware and platform details
 - more powerful primitives and interfaces
 - leaving difficult task to intermediaries (compilers, optimizers, automatic load balancing, automatic data partitioning and allocation, etc.)
 - reducing the number of programming errors
 - reducing the development and maintenance cost of the applications developed by facilitating their portability
- Middleware is primarily a set of programming abstractions developed to facilitate the development of complex distributed systems
 - to understand a middleware platform one needs to understand its programming model
 - from the programming model the limitations, general performance, and applicability of a given type of middleware can be determined in a first approximation
 - the underlying programming model also determines how the platform will evolve and fare when new technologies evolve

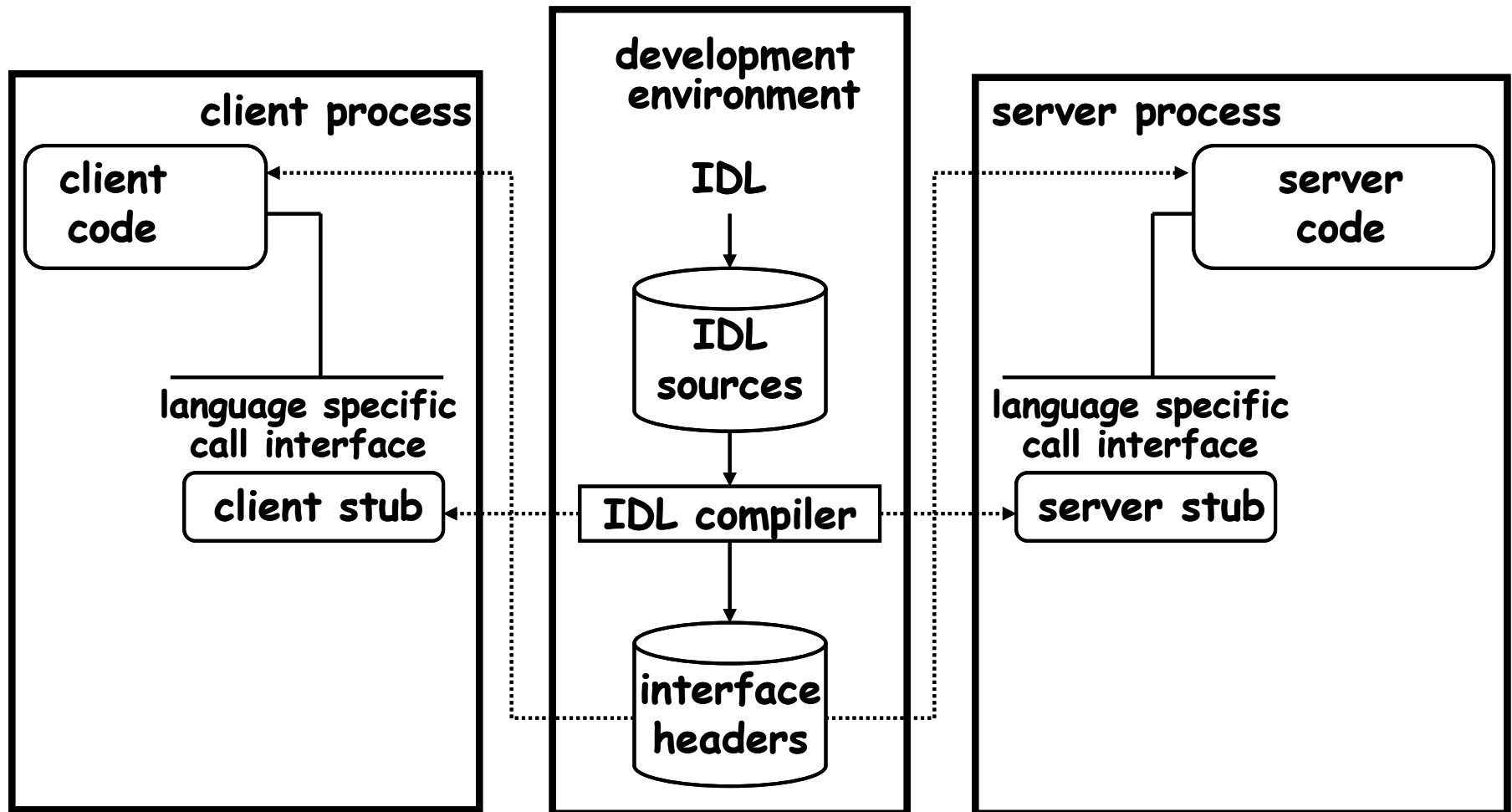


Middleware as Infrastructure

- As the programming abstractions reach higher and higher levels, the underlying infrastructure implementing the abstractions must grow accordingly
 - Additional functionality is almost always implemented through additional software layers
 - The additional software layers increase the size and complexity of the infrastructure necessary to use the new abstractions
- The infrastructure is also intended to support additional functionality that makes development, maintenance, and monitoring easier and less costly
 - RPC => transactional RPC => logging, recovery, advanced transaction models, language primitives for transactional demarcation, transactional file system, etc.
 - The infrastructure is also there to take care of all the non-functional properties typically ignored by data models, programming models, and programming languages: performance, availability, recovery, instrumentation, maintenance, resource management, etc.



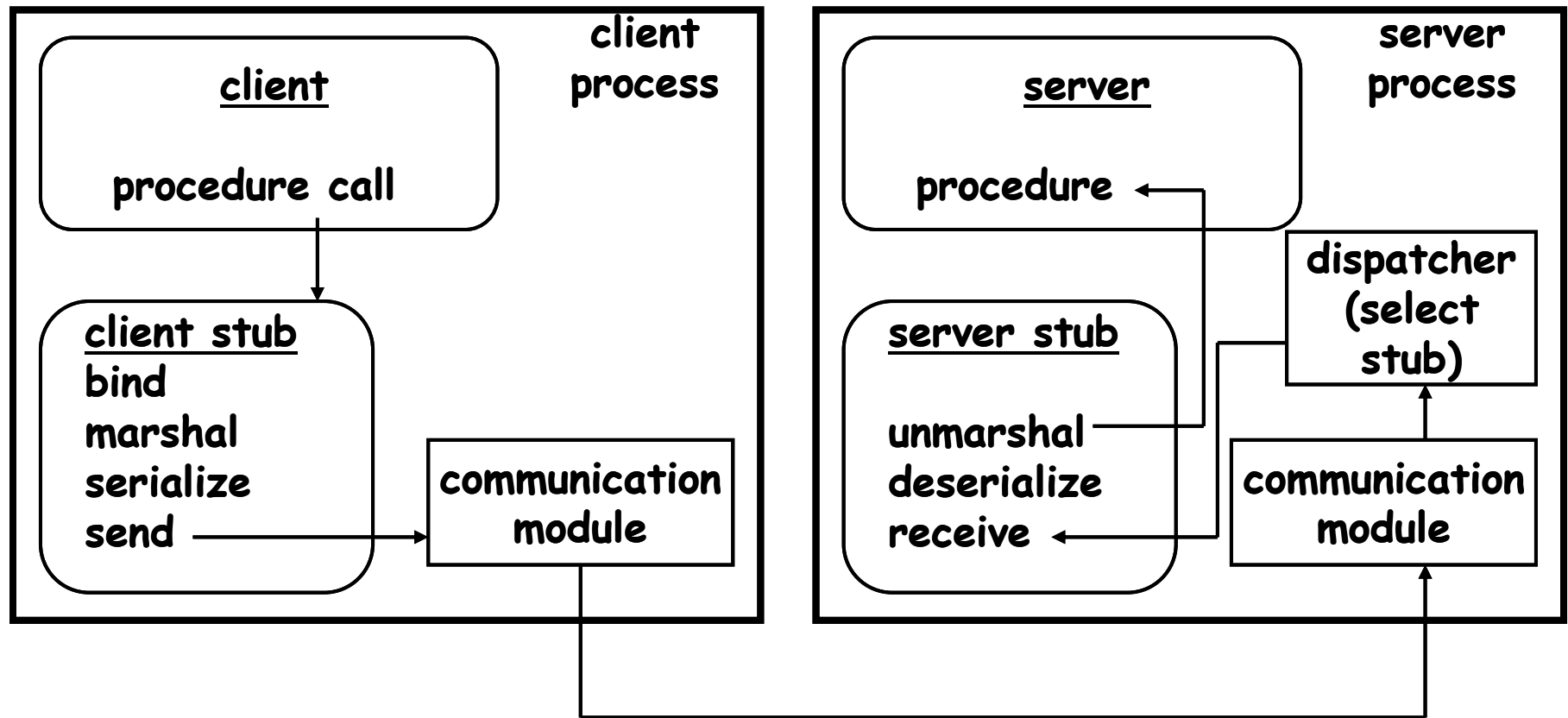
Basic Middleware: RPC (1)



Basic Middleware: RPC (2)

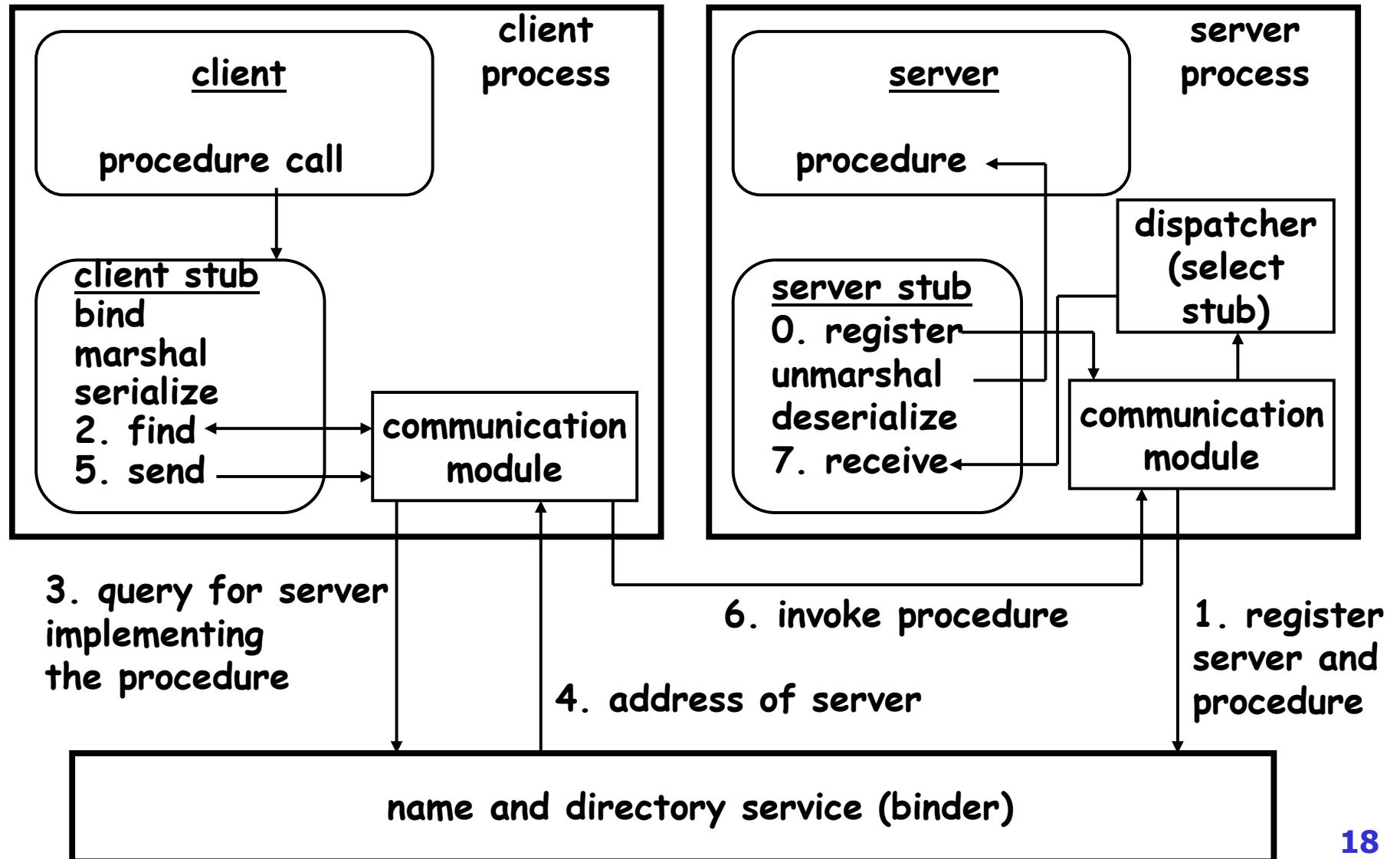


SAPIENZA
UNIVERSITÀ DI ROMA



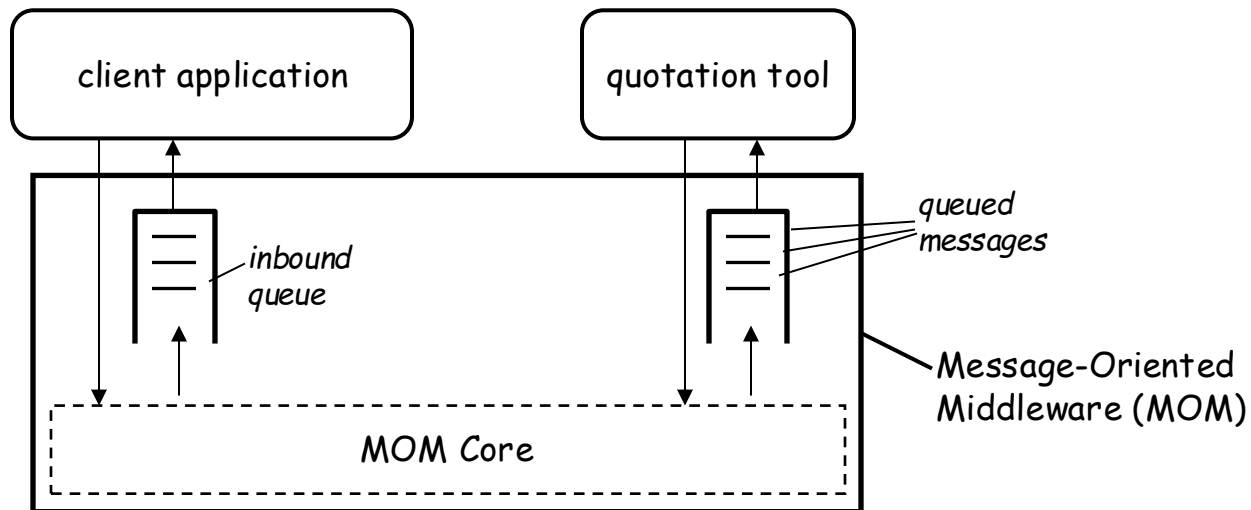


Basic Middleware (3)



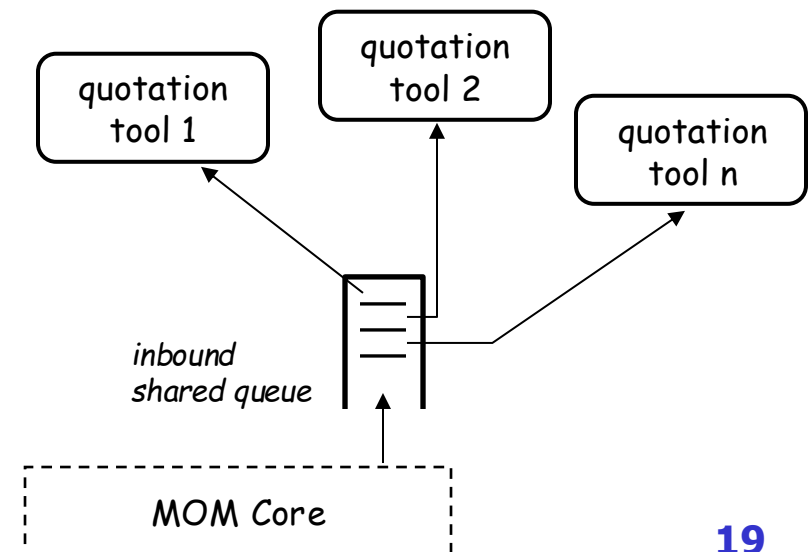


Message Oriented Middleware



The queuing model

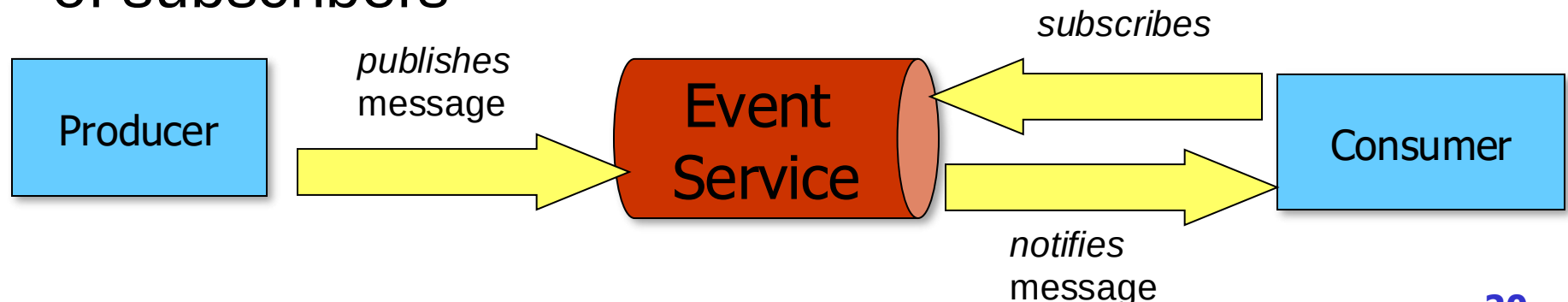
(Primitive) load
balancing with
shared queues





Publish/Subscribe

- A many-to-many anonymous interaction
- The participants are divided into two groups
 - publisher: producers, send messages
 - subscriber: consumers, express interest in certain categories of messages
- Communication takes place through a central entity (event service) that receives messages from publishers and cross-check them with the interests of subscribers





Publish/Subscribe

- **Topic-based:** information is divided into topics (topic or subject)
 - subscriptions and publications by topic
 - ideal logical channel that connects a publisher to all subscribers
- **Content-based:** filters can be used for a more accurate selection of information to be received (message content)
- The notification can be made in two ways
 - push: the subscribers are invoked in callback, using a reference communicated at the time of subscription
 - pull: the subscribers poll the event service when they need messages



Example

```
Message QuoteRequest {  
    QuoteRefereneceNumber: Integer  
    Customer: String  
    Item: String  
    Quantity: String  
    RequestedDeliveryDate: Timestamp  
    DeliveryAddress: String  
}
```

With P&S topic-based a subscriber registers its interest for messages of type `QuoteRequest` (~ interest for the "class")

With P&S content-based a subscriber registers its interest for messages of type `QuoteRequest` such that

`RequestedDeliveryDate` before June 30th, 2004 AND `Quantity` > 1000

You can write expressions that allow you to filter all the messages of a type with respect to the attribute values (selection of the "instance"). The expressive power of the subscription language becomes important (a sort of query language)

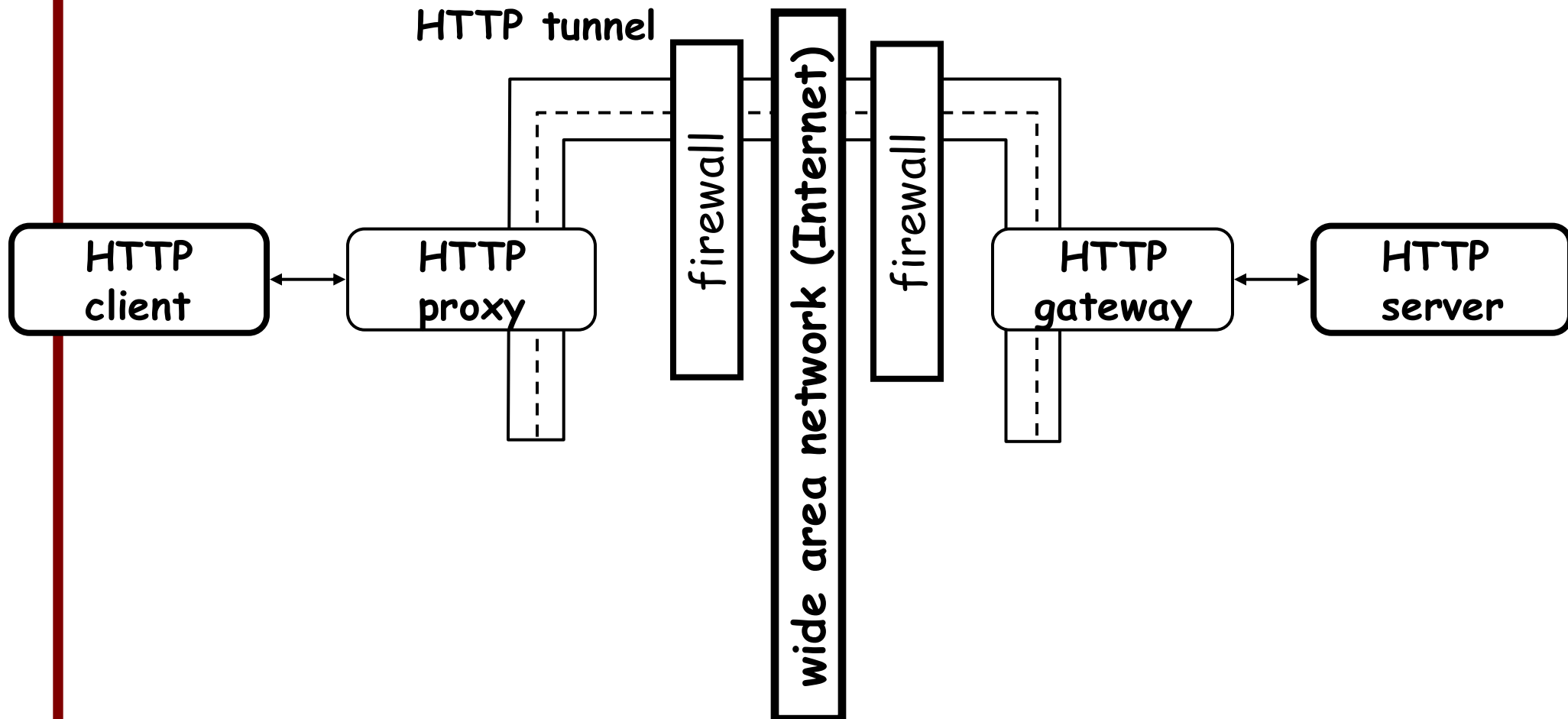


SAPIENZA
UNIVERSITÀ DI ROMA

MIDDLEWARE ON THE INTERNET

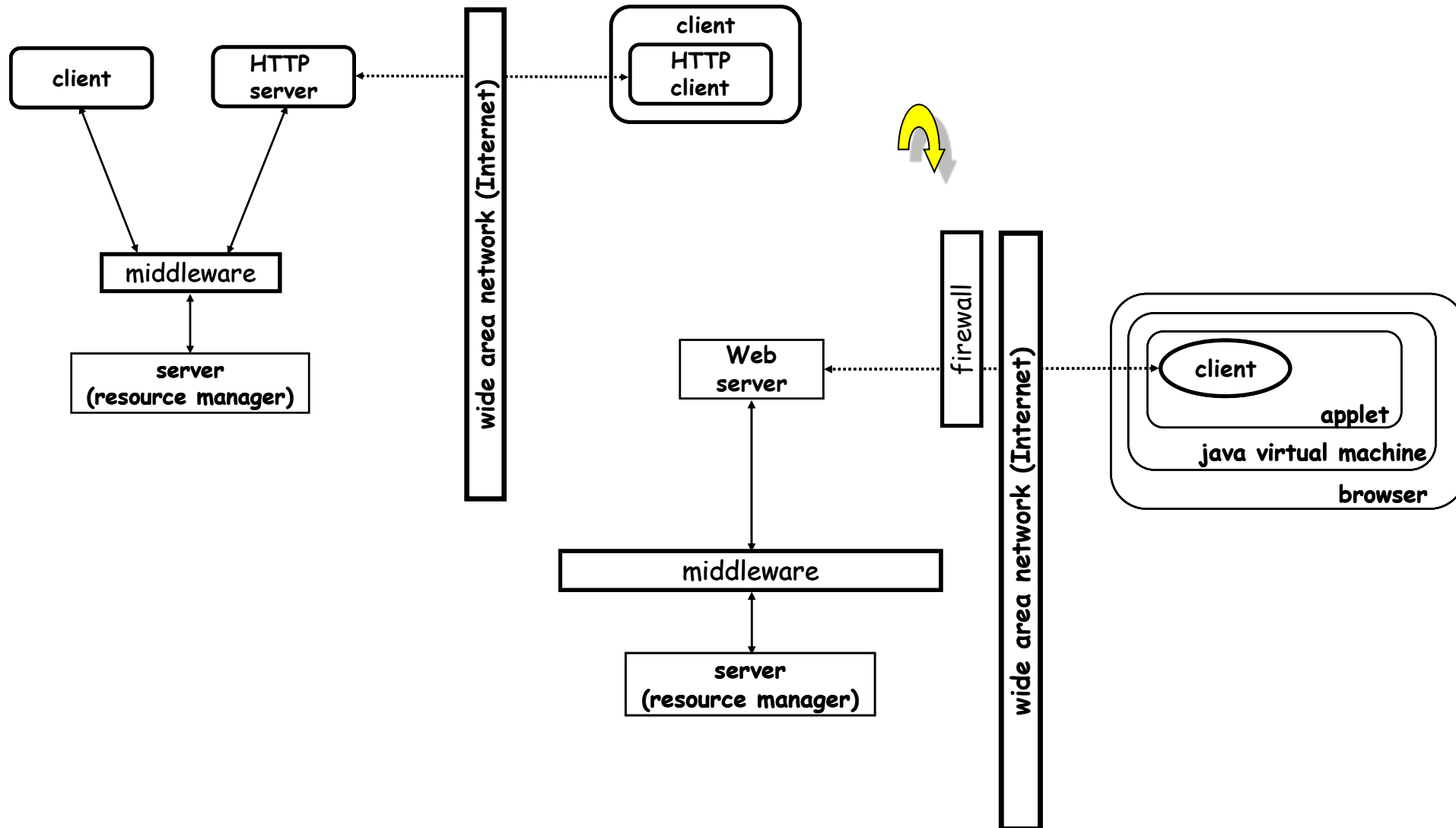


Proxy and Firewall



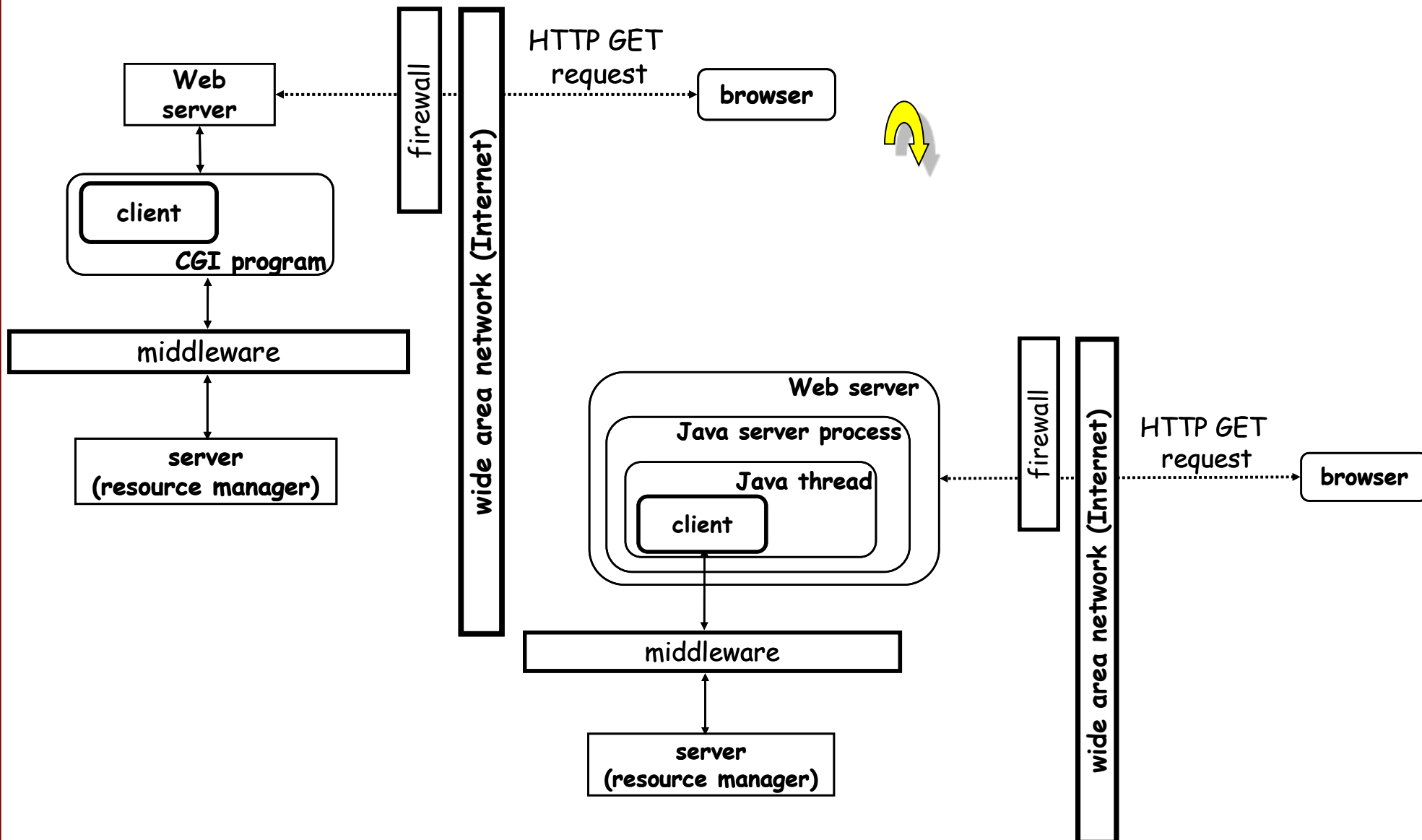


3-tier on the Web: Support for Remote Clients



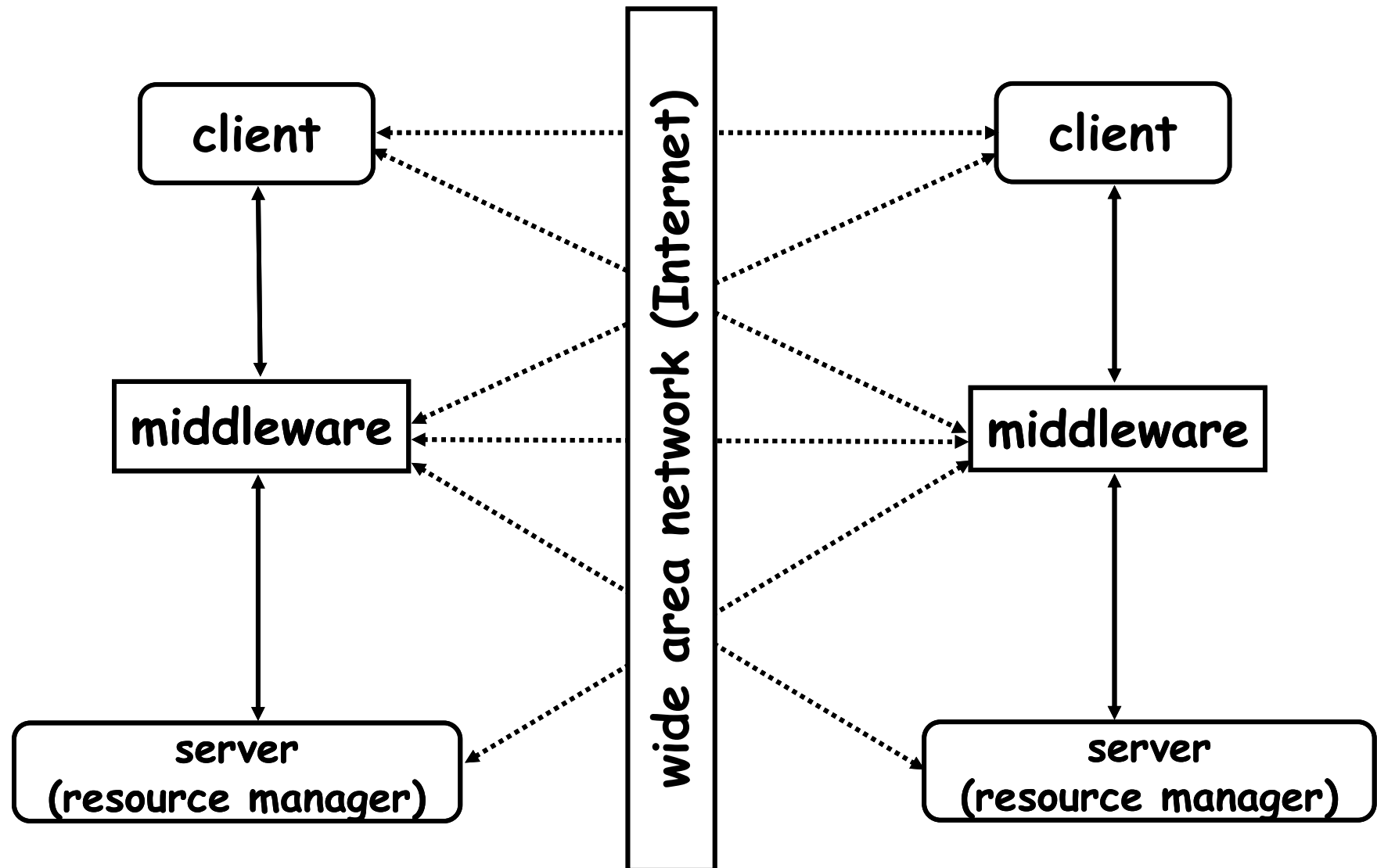


3-tier on the Web: Server Pages





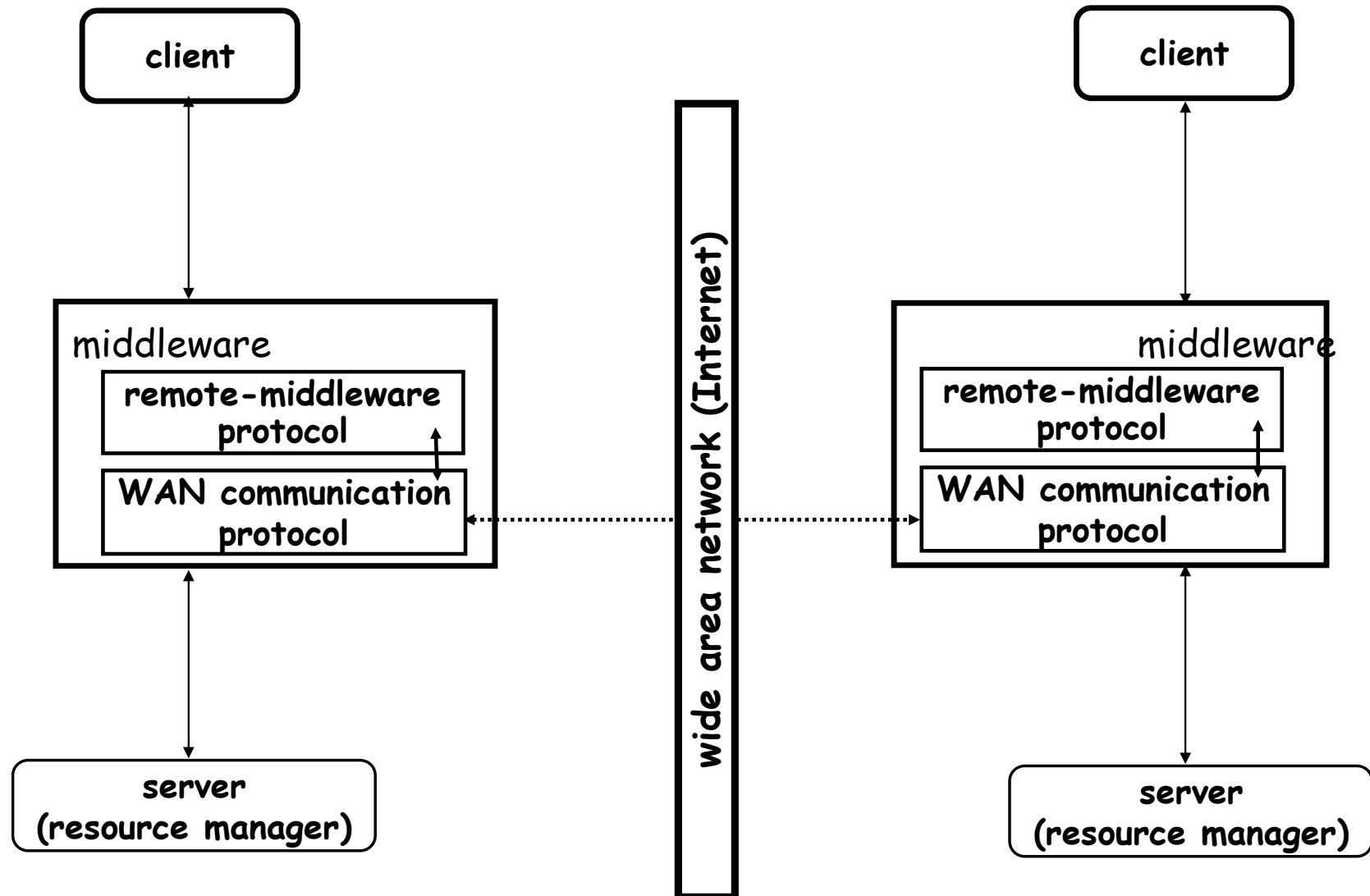
Integration Across the Internet



Integration M2M (middleware-to-middleware)



...





... may require HTTP Tunnelling/Support

